

CSE 2217: Data Structure and Algorithms

Introduction

Proof By Induction

Asymptotic notation

What is an Algorithm?

- An algorithm is a sequence of computational steps that solves a well-specified computational problem.
 - An algorithm is said to be **correct** if, for every input instance, it halts with the correct output
 - An **incorrect** algorithm might not halt at all on some input instances, or it might halt with other than the desired output.

What is a program?

- A program is the expression of an algorithm in a programming language
- a set of instructions which the computer will follow to solve a problem



Where We're Going

- Examine methods of analyzing algorithm correctness and efficiency
 - Lower bound techniques
 - O , Ω and Θ notations for best/worst/average case analysis
- Decide whether some problems have no solution in reasonable time
 - List all permutations of n objects (takes $n!$ steps)
 - Travelling salesman problem (Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city?)
- Investigate memory usage as a different measure of efficiency

The study of Algorithm

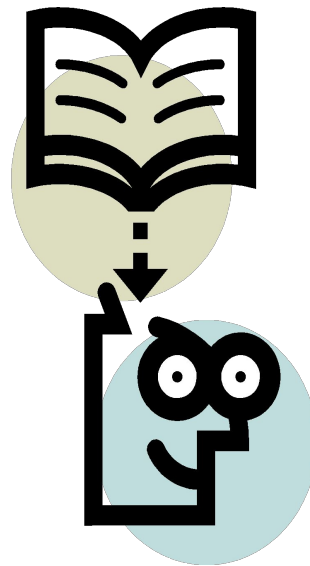
- How to devise algorithms
- How to express algorithms
- How to validate algorithms
- How to analyze algorithms
- How to test a program

Importance of Analyzing Algorithm

- Need to recognize limitations of various algorithms for solving a problem
- Need to understand relationship between problem size and running time
 - When is a running program not good enough?
- Need to learn how to analyze an algorithm's running time without coding it
- Need to recognize bottlenecks in code as well as which parts of code are easiest to optimize

Why do we analyze about them?

- understand their behavior, and (Job -- Selection, performance, modify)
- improve them. (Research)



What do we analyze about them?

- Correctness
 - Does the input/output relation match algorithm requirement?
- Amount of work done (aka complexity)
 - Basic operations to do task
- Amount of space used
 - Memory used

What do we analyze about them?

- **Simplicity, clarity**
 - Verification and implementation.
- **Optimality**
 - Is it impossible to do better?



Complexity

- The complexity of an algorithm is simply the amount of work the algorithm performs to complete its task.



RAM model

- has one processor
- executes one instruction at a time
- each instruction takes "unit time"
- has fixed-size operands, and
- has fixed size storage (RAM and disk).



The Selection Problem (1/2)

- Problem: given a group of n numbers, determine the k^{th} largest
- Algorithm 1
 - Store numbers in an array
 - Sort the array in descending order
 - Return the number in position k

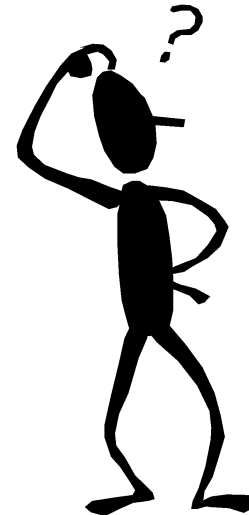


The Selection Problem(2/2)

- Algorithm 2

- Store first k numbers in an array
- Sort the array in descending order
- For each remaining number, if the number is larger than the k^{th} number, insert the number in the correct position of the array
- Return the number in position k

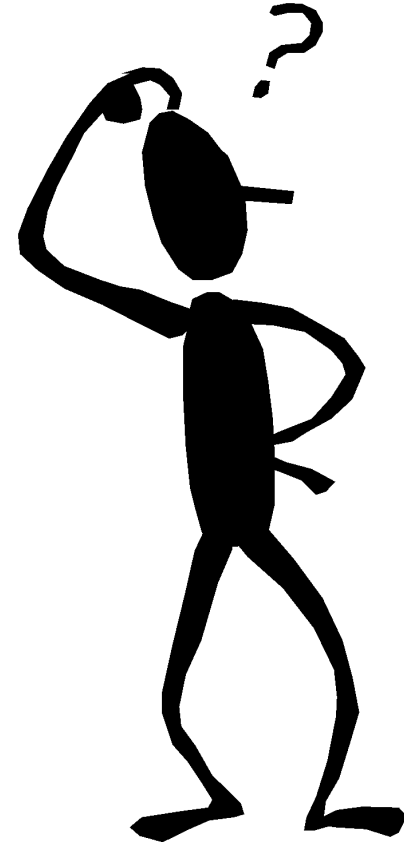
Which algorithm is better?



Which algorithm is better?

**The algorithms are correct,
but which is the best?**

- Measure the running time (number of operations needed).
- Measure the amount of memory used.
- Note that the running time of the algorithms increase as the size of the input increases.

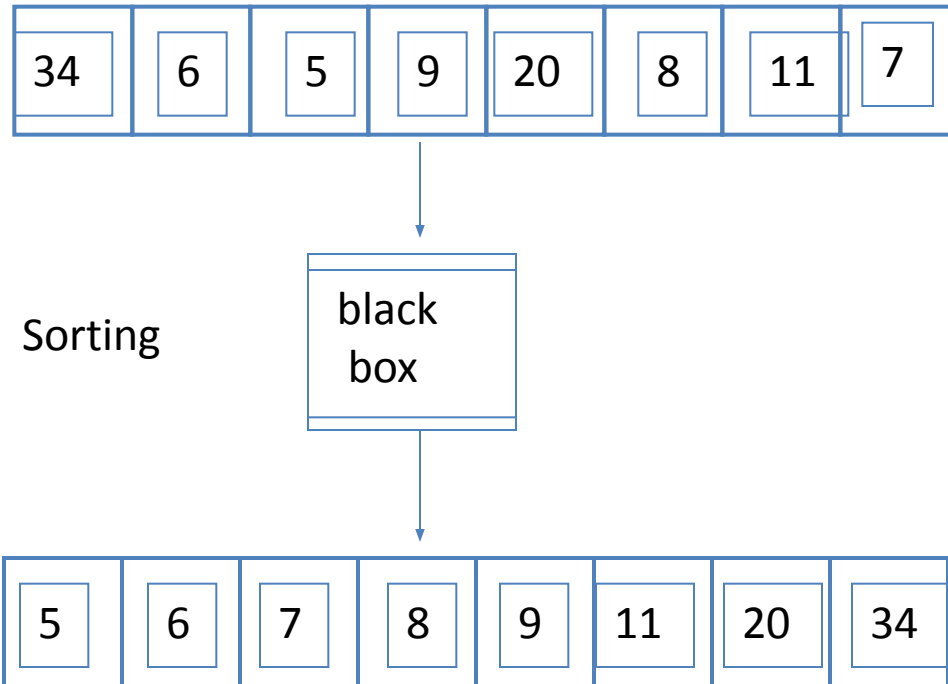


Define Problem

- **Problem:**
 - Description of Input-Output relationship
- **Algorithm:**
 - A sequence of computational step that transform the input into the output.
- **Data Structure:**
 - An organized method of storing and retrieving data.
- **Our task:**
 - Given a problem, design a *correct* and *good* algorithm that solves it.

Example Algorithm

Sort the input in increasing order. Return the first element of the sorted data.



What do we need?

Correctness: Whether the algorithm computes the correct solution for **all** instances

Efficiency: Resources needed by the algorithm

1. Time: Number of steps.
2. Space: amount of memory used.

Measurement “model”: Worst case, Average case and Best case.

Methods of Proof

- **Proof by Contradiction**
 - Assume a theorem is false; show that this assumption implies a property known to be true is false -- therefore original hypothesis must be true
- **Proof by Counterexample**
 - Use a concrete example to show an inequality cannot hold
- **Mathematical Induction**
 - Prove a trivial base case, assume true for k , then show hypothesis is true for $k+1$
 - Used to prove recursive algorithms

Proof by Contradiction

- In a proof of contradiction, one assumes that the hypothesis and the negation of the conclusion are true and then to deduce some contradiction from these assumptions.
- Example: Prove that “If a number added to itself gives itself, then the number is 0.”
 - The hypothesis (P) is $x + x = x$ and the conclusion (Q) is $x = 0$. Hence, the hypotheses for the proof by contradiction are:
 - $x + x = x$ and $x \neq 0$ (the negation of the conclusion)
 - Then $2x = x$ and $x \neq 0$, hence dividing both sides by x , the result is $2 = 1$, which is a contradiction. Hence, $(x + x = x) \rightarrow (x = 0)$.

Proof by Counterexample

Example: Prove that “For every positive integer n , $n! \leq n^2$.”

- Start testing some cases say, $n = 1, 2, 3$ etc.
- It might seem like it is true for some cases but how far do you test, say $n = 4$.
- We get $n! = 24$ and $n^2 = 16$ which is a counterexample for this theorem. Hence, even finding a single case that doesn't satisfy the condition is enough to disprove the theorem.

Proof By Induction

- Claim: $S(n)$ is true for all $n \geq k$
- Basis:
 - Show formula is true when $n = k$
- Inductive hypothesis:
 - Assume formula is true for an arbitrary n
- Step:
 - Show that formula is then true for $n+1$

Induction Example: Gaussian Closed Form

- Prove $1 + 2 + 3 + \dots + n = n(n+1) / 2$

- Basis:

- If $n = 0$, then $0 = 0(0+1) / 2$

- Inductive hypothesis:

- Assume $1 + 2 + 3 + \dots + n = n(n+1) / 2$

- Step (show true for $n+1$):

$$\begin{aligned} 1 + 2 + \dots + n + n+1 &= (1 + 2 + \dots + n) + (n+1) \\ &= n(n+1)/2 + n+1 = [n(n+1) + 2(n+1)]/2 \\ &= (n+1)(n+2)/2 = (n+1)(n+1 + 1) / 2 \end{aligned}$$

Induction Example:

Geometric Closed Form

- Prove $a^0 + a^1 + \dots + a^n = (a^{n+1} - 1)/(a - 1)$ for all $a \neq 1$